



Secure Network Application Development Using TCP

Internship Report

Submitted by

Vidit Suryavanshi
Bachelor of Technology Computer Science & Engineering
(Cyber Security)
Oriental College of Technology Bhopal, (M.P)
JUN-JULY, 2026

1. Objective

The objective of this assignment was to enhance the GUI-based multi-client TCP chat application from Assignment 6 by adding practical security features. The upgraded system keeps the same client-server model, but now it enforces authentication, password hashing, duplicate login prevention, input validation, temporary login blocking, session timeout, and secure logging. The focus is on application-layer security and safe user handling rather than heavy cryptography.

2. Security Features Implemented

- Username and password based authentication.
- Password storage using SHA-256 hashes in users.json.
- Duplicate login prevention for the same user account.
- Validation for usernames, passwords, messages, and commands.
- Temporary blocking after five consecutive failed login attempts.
- Logout support and inactivity-based session timeout.
- Secure logging without storing plaintext passwords.
- Private messaging, broadcast messaging, and online user refresh.

3. System Architecture

The application uses a client-server architecture over TCP port 5000. In Mininet topology single,5, the server runs on one host and multiple client instances run on the remaining hosts. The Tkinter client connects to the server, sends login credentials, receives authentication responses, and then switches to the chat interface. A background receiver thread keeps the GUI responsive while messages and user list updates are processed.

Component	Role in the System
server.py	Authentication, message routing, blocking, timeout, and logging.
client_gui.py	Tkinter login screen, chat window, and user interaction.
users.json	SHA-256 hashed credentials for registered users.
chat_history.csv	Message history for broadcast and private messages.
security_log.txt	Security audit trail for login, timeout, and disconnect events.

4. Implementation Details

4.1 Authentication and Password Storage

The server first validates the username format and then checks whether the account exists in `users.json`. Passwords are hashed with `hashlib.sha256()` before comparison, so the server never relies on plaintext password storage. This is the most important control in the assignment because it prevents exposed credentials from being reused directly from disk.

4.2 Duplicate Login Prevention

After a user is authenticated, the server stores that username in an in-memory logged-in set. If the same username tries to connect again while the first session is active, the second attempt is rejected with a duplicate-login error. This stops session hijacking and prevents one account from being used in two places at once.

4.3 Input Validation and Failed Login Blocking

The implementation rejects empty passwords, passwords shorter than six characters, invalid usernames, unsupported slash commands, and messages that exceed the configured maximum length. Failed login attempts are counted per user. After five failures, the account is blocked for 60 seconds and the event is written to the security log.

4.4 Session Management and Secure Logging

Each active client session tracks last activity time. If a user stays idle beyond the configured timeout, the server closes the session and records a `SESSION TIMEOUT` event. The server also logs `LOGOUT`, `DISCONNECTED`, `LOGIN SUCCESS`, `LOGIN BLOCKED`, and `DUPLICATE LOGIN BLOCKED` events. The log file is useful for forensic review and troubleshooting.

5. Testing and Results

The screenshots below were used to verify each security control and to confirm that the client and server behave correctly under normal and abnormal conditions. The order follows the actual test flow: failed login cases, successful login, duplicate access prevention, session handling, normal messaging, and traffic verification in Wireshark.

5.1 Authentication Validation

The first set of tests checks whether the server correctly rejects bad credentials and weak input before allowing access to the chat system.

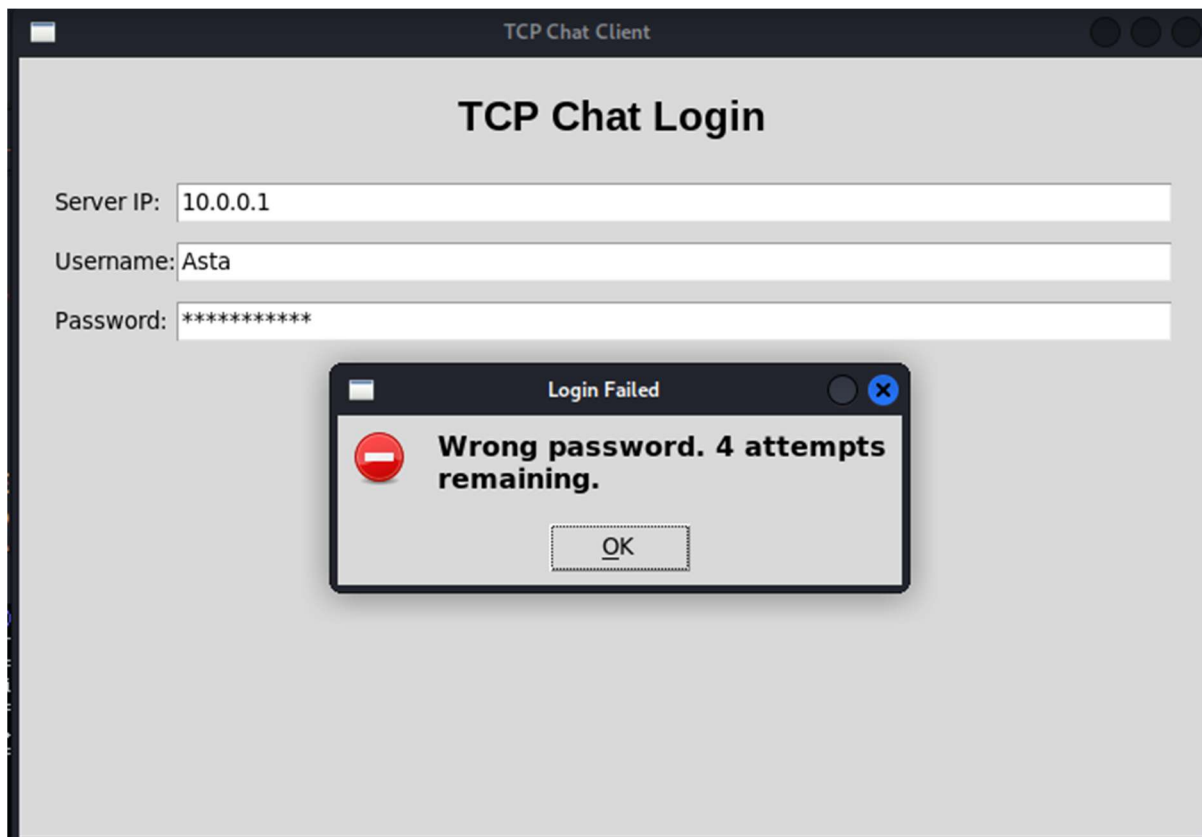


Figure 1: Wrong password attempt showing remaining tries.

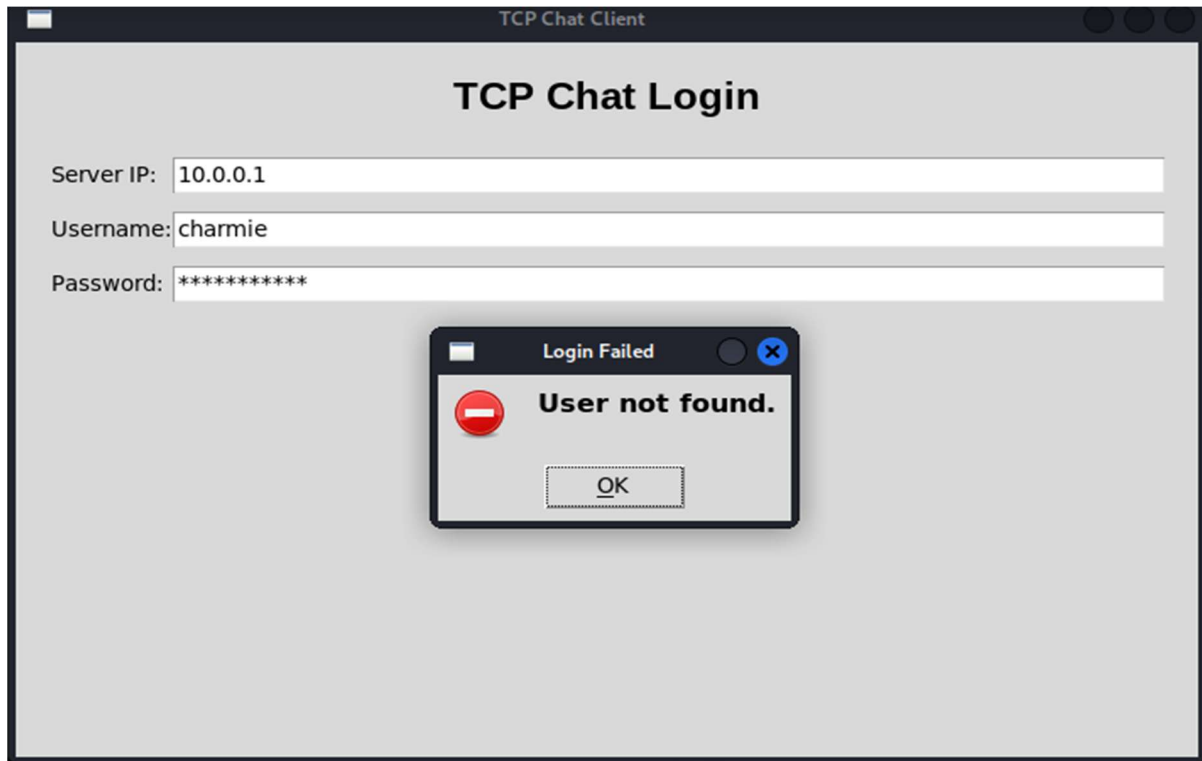


Figure 2: Unknown user rejected during login.

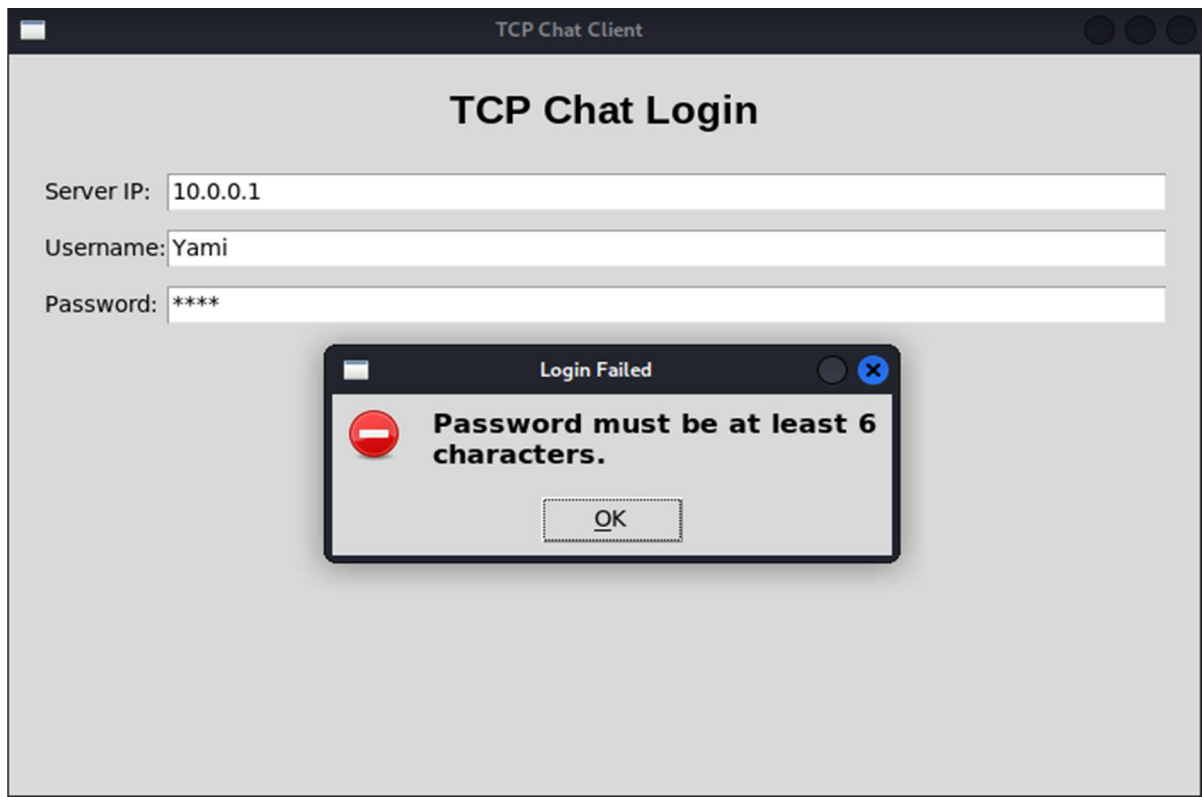


Figure 3: Password length validation on the client-side/server response.

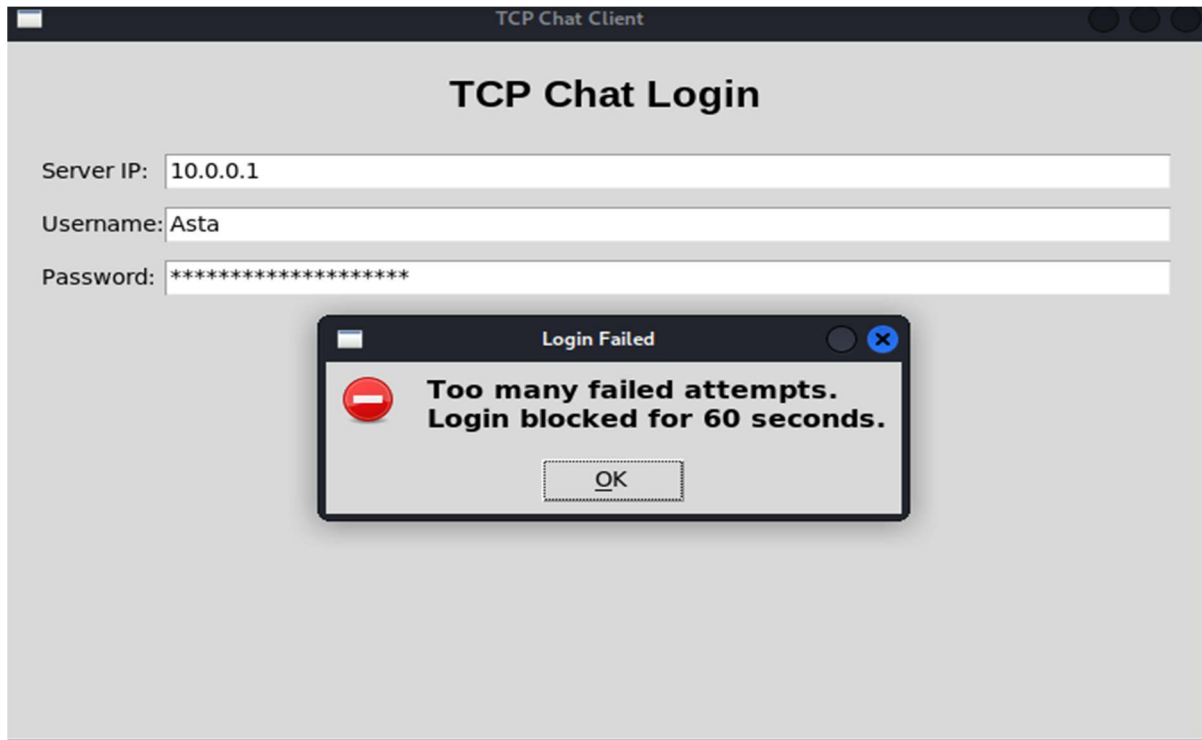


Figure 4: Account blocked after repeated failed login attempts.

5.2 Successful Login and Duplicate Login Control

Once valid credentials are supplied, the user reaches the main chat window. The duplicate login test confirms that the same username cannot establish a second active session.



Figure 5: Successful login and main chat interface.

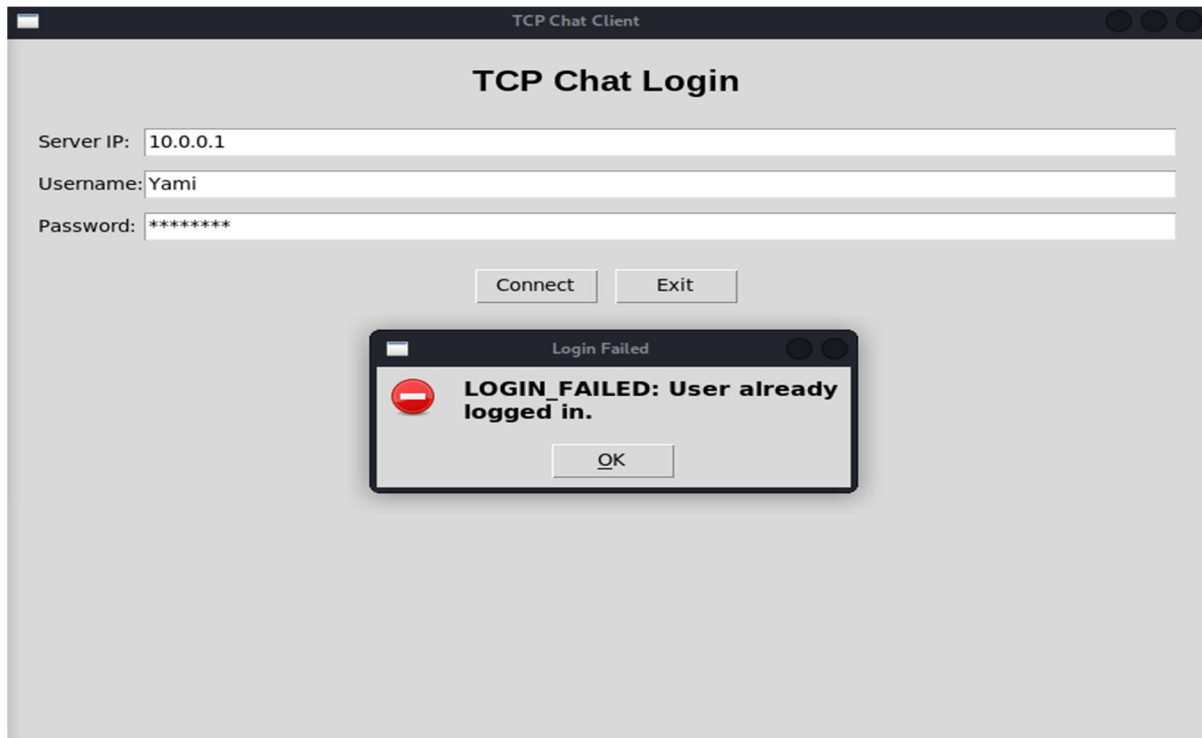


Figure 6: Duplicate login blocked for an already active user.

5.3 Session Management and Messaging

These screenshots show the server closing idle sessions, updating the online user list, and continuing to support broadcast and private chat after authentication.

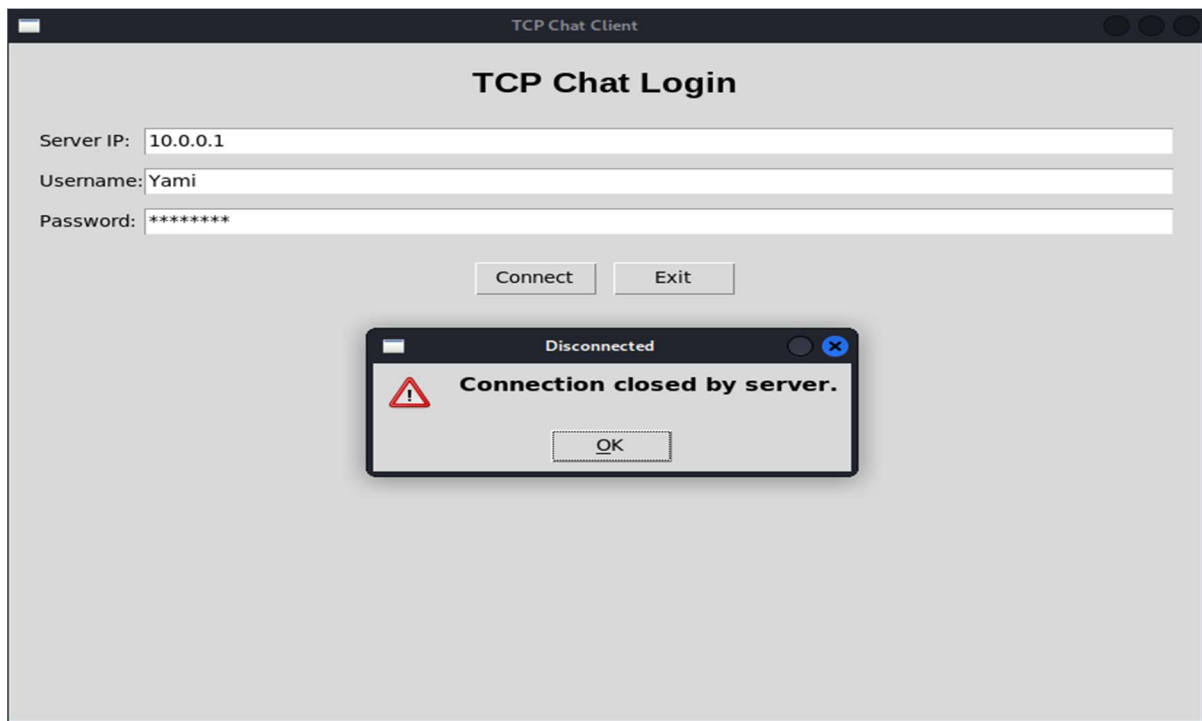


Figure 7: Session disconnected by the server after inactivity.

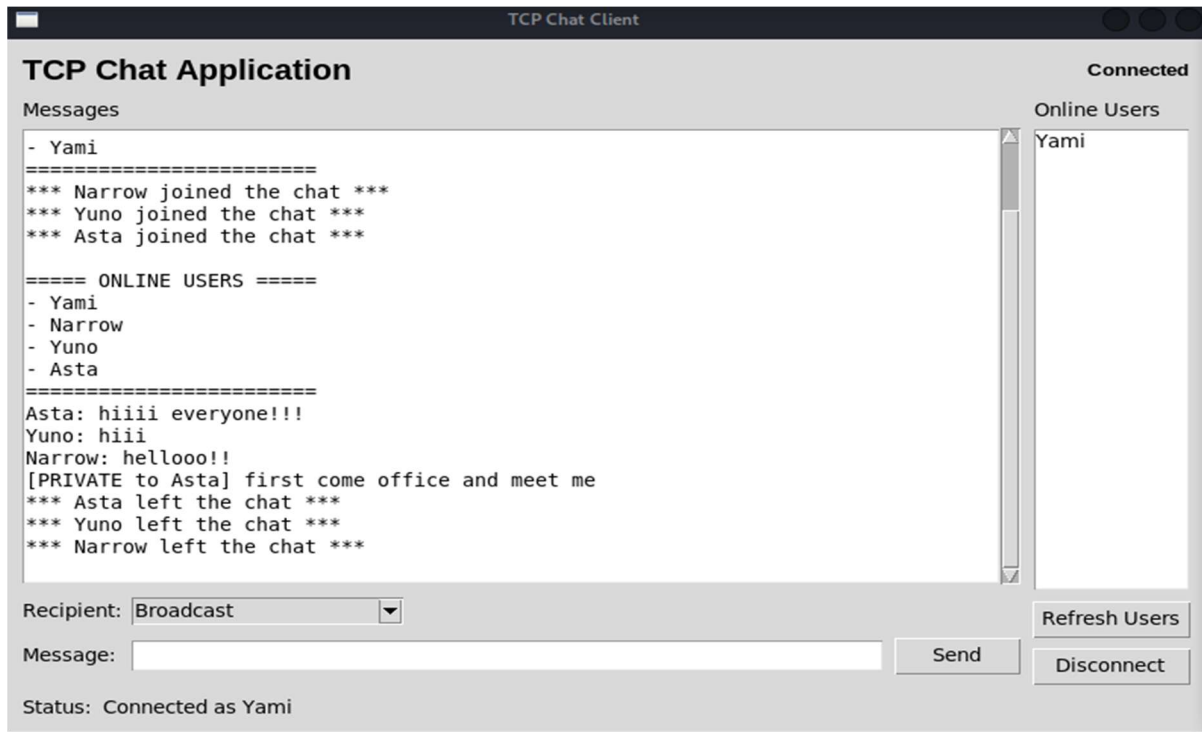


Figure 8: Authenticated chat session with broadcast and private messaging evidence.

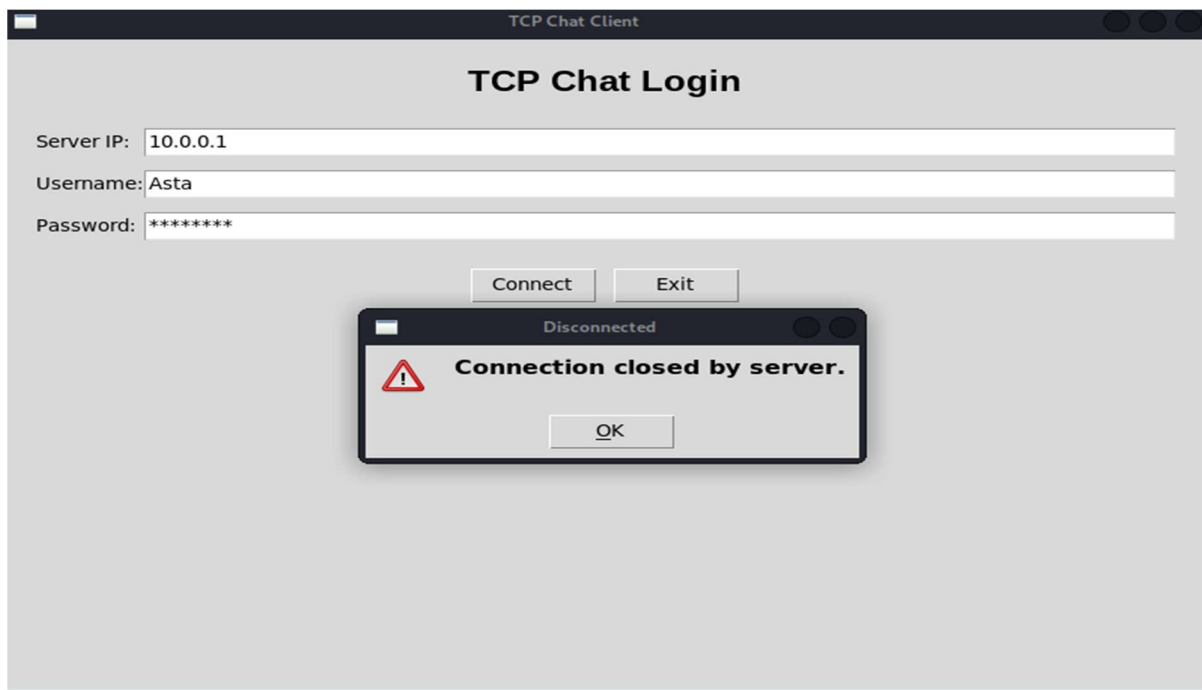


Figure 9: Another server-side disconnect example after session termination.

5.4 Wireshark Verification

Wireshark captures taken on `tcp.port == 5000` verify the TCP handshake, failed login flow, authenticated data exchange, and disconnect behavior. The captures provide independent evidence that the application really sends traffic over the expected TCP connection.

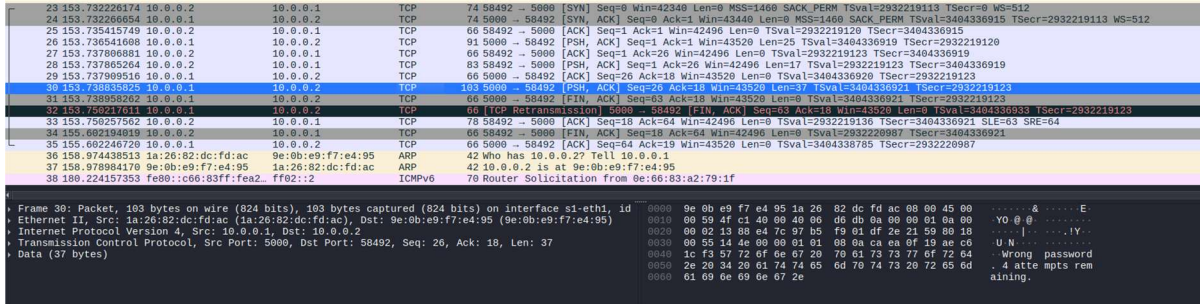


Figure 10: Wireshark capture of failed login traffic on TCP port 5000.

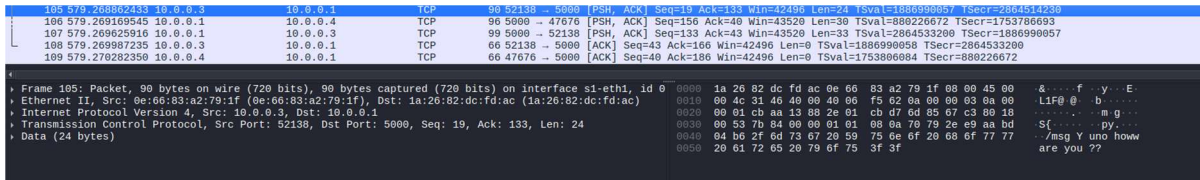


Figure 11: Wireshark capture of authenticated message traffic.

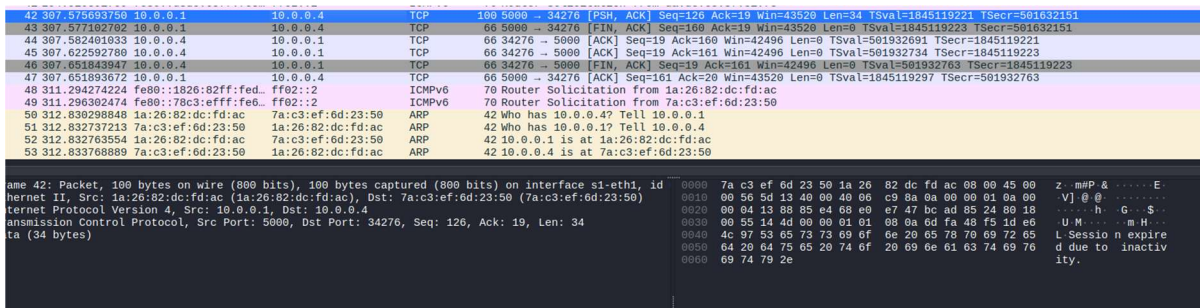


Figure 12: Wireshark capture of logout or timeout-related traffic.

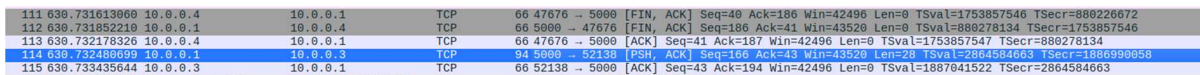


Figure 13: Additional FIN/ACK disconnect traffic evidence.

6. Conclusion

This assignment successfully upgraded the earlier GUI TCP chat application into a more secure system. The final solution protects accounts with hashed password storage, rejects weak or invalid login attempts, blocks repeated failures, prevents duplicate logins, and logs security events for later review. The GUI remains responsive, the chat features still work, and the Wireshark evidence confirms that the application behaves as expected on the network.